



GitHub Actions

Introduction

GitHub (<https://github.com>) est bien plus qu'une simple plateforme d'hébergement et de partage de code.

Avec GitHub Actions, les développeurs ont accès à un ensemble d'outils puissants et flexibles pour **automatiser leurs workflows**, de l'intégration continue (CI) et du déploiement continu (CD) aux tâches d'automatisation personnalisées.

Introduction

Définitions

L'intégration continue (CI, *Continuous Integration*) est une pratique de développement logiciel qui consiste à **intégrer fréquemment les changements** de code dans un dépôt central, puis à exécuter **automatiquement des tests et vérifications**.

Son utilité principale est d'améliorer la qualité du logiciel et de réduire les risques liés au développement.
=> le code est testé

La **Livraison Continue (CD – Continuous Delivery)** est une pratique qui complète l'intégration continue en automatisant tout le processus menant *jusqu'au seuil du déploiement*.

Elle garantit que l'application peut être mise en production **à tout moment**, de façon sûre et rapide, mais avec une validation humaine finale. => le code *prêt à être déployer*

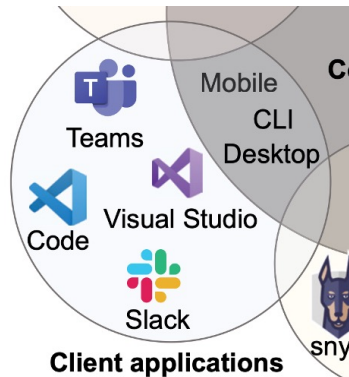
Ecosystème de GitHub

Planning and tracking



Planification et suivi — Outre les tickets (issues) et les jalons, GitHub propose **GitHub Discussions**, un forum dédié à la collaboration sur les idées.

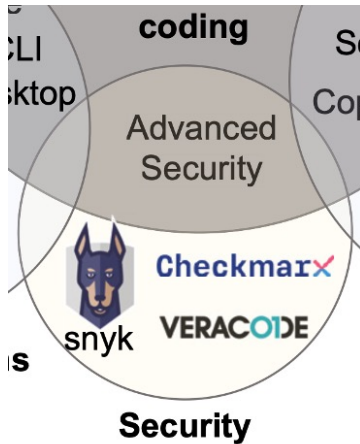
De plus, **GitHub Projects** est une solution de planification flexible entièrement intégrée aux tickets et aux pull requests, qui prend en charge les backlogs imbriqués, les tableaux et les feuilles de route.



Applications client — GitHub fournit un éditeur de code complet accessible

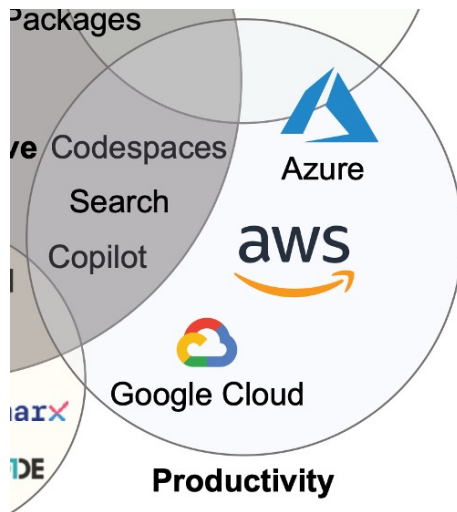
directement dans le navigateur. Il propose également des applications mobiles pour les plateformes iOS et Android, permettant aux équipes de collaborer où qu'elles se trouvent. De plus, une application de bureau multiplateforme et une interface de ligne de commande extensible (CLI) sont disponibles.

Ecosystème de GitHub



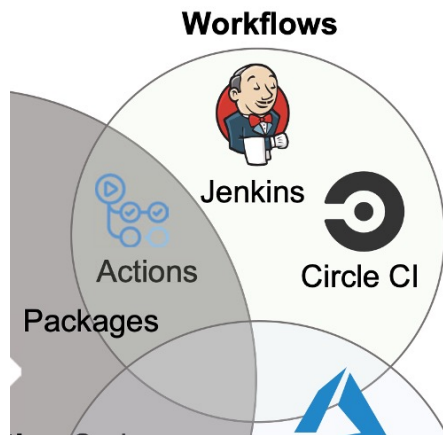
Sécurité — GitHub fournit une solution complète pour garantir la sécurité de la chaîne logistique des logiciels, qui comprend plusieurs fonctionnalités clés. Par exemple, il génère des nomenclatures logicielles pour suivre tous les composants inclus dans votre logiciel. Et grâce à sa fonctionnalité Dependabot, GitHub peut vous alerter dès qu'une vulnérabilité est détectée dans l'une des dépendances que vous utilisez. De plus, GitHub peut analyser votre référentiel pour détecter des secrets, et dispose d'un moteur d'analyse de code sophistiqué appelé CodeQL.

Ecosystème de GitHub



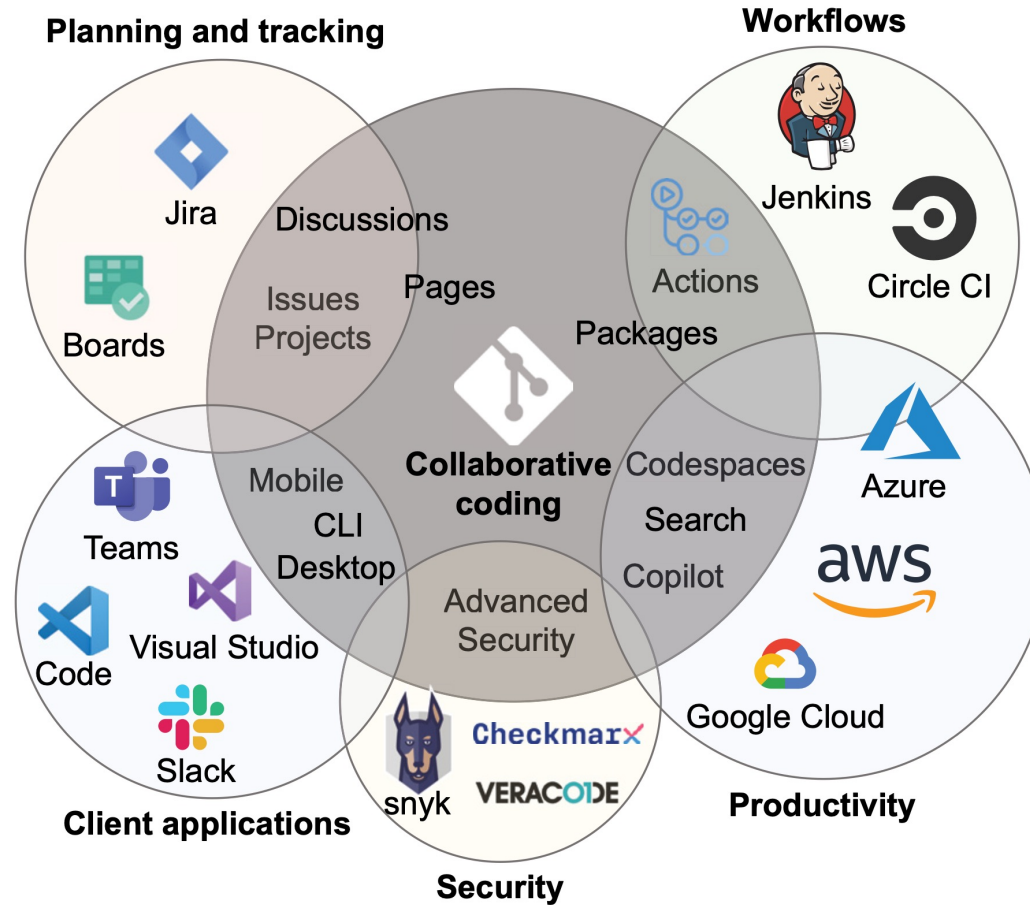
Productivité des développeurs — Dans GitHub, les développeurs peuvent rapidement créer un environnement de développement personnalisé et conteneurisé à l'aide de **GitHub Codespaces**. Cela permet aux nouveaux développeurs d'être immédiatement productifs. De plus, **Copilot**, un assistant alimenté par l'IA, peut générer du code en fonction du contexte des commentaires ou d'autres codes. Cela peut augmenter considérablement la productivité, avec des gains pouvant atteindre 50 % selon certains rapports.

Ecosystème de GitHub



Workflows et CI/CD — Dans le monde de l'intégration continue et de la livraison continue(CI/CD), GitHub est une plateforme populaire largement prise en charge par la plupart des outils CI/CD disponibles sur le marché. De plus, GitHub offre une intégration sécurisée avec tous les principaux fournisseurs de cloud pour les workflows CI/CD utilisant Open ID Connect (OIDC). Cela garantit une expérience sécurisée et rationalisée aux développeurs qui s'appuient sur des services basés sur le cloud. De plus, **GitHub Packages** est équipé d'un registre de paquets robuste qui prend en charge un large éventail de formats de paquets, offrant ainsi aux développeurs un outil puissant et polyvalent pour gérer et distribuer leurs paquets de code.

Ecosystème de GitHub



GitHub Actions et workflows

```
my-first-action / .github / workflows / hello.yml in main
Edit Preview
1  name: CI
2
3  on:
4    push:
5      branches: [ "main" ]
6    pull_request:
7      branches: [ "main" ]
8
9    workflow_dispatch:
10
11 jobs:
12   build:
13     runs-on: ubuntu-latest
14     steps:
15       - uses: actions/checkout@v4
16
17       - name: Run a one-line script
18         run: echo Hello, world!
19
20       - name: Run a multi-line script
21         run: |
22           echo Add other actions to build,
23           echo "Nom du dépôt : ${github.event.repository.name}"
```

workflow : hello.yml

Événements qui déclenchent le workflow

Regroupement de tâches

Le type de VM sur laquelle sera exécutée les tâches

Étapes

Utilisation d'une Action

Exécution d'une commande : echo

Syntaxe des workflows-Déclencheurs

Nom du workflow

name: nom du workflow

Déclencheurs

Les déclencheurs (triggers) sont le moyen de spécifier les critères d'exécution du workflow.

Il existe trois catégories de déclencheurs :

- Les déclencheurs automatiques (basé sur un événement GitHub, webhook)
- Les déclencheurs programmés (planifiés)
- Les déclencheurs manuels

Tous les déclencheurs suivent la clé **on:** dans le fichier de workflow.

Syntaxe des workflows-Déclencheurs

Les déclencheurs automatiques

Le trigger **push** dans un workflow GitHub Actions sert à **déclencher automatiquement un workflow dès qu'un push est effectué sur le dépôt.**

```
on : push
```

Le trigger **pullrequest** dans un workflow GitHub Actions sert à **déclencher automatiquement un workflow dès qu'un développeur demande un « pull request ».**

```
on :  
  push:  
  pullrequest:
```

Syntaxe des workflows-Déclencheurs

Les déclencheurs automatiques

Le trigger **push** dans un workflow GitHub Actions sert à **déclencher automatiquement un workflow dès qu'un push est effectué sur le dépôt.**

```
on : push
```

Le trigger **pullrequest** dans un workflow GitHub Actions sert à **déclencher automatiquement un workflow dès qu'un développeur demande un « pull request ».**

```
on :  
  push:  
  pullrequest:
```

Syntaxe des workflows-Déclencheurs

Les déclencheurs automatiques - filtres

Il existe de type de filtre :

- Filtres par branche
- Filtres par chemin

Les **filtres par branches** fonctionnent avec différents triggers (`push`, `pull_request`, etc.).
Ils permettent de cibler **quelle branche déclenche réellement le workflow**.

```
on:  
  push:  
    branches:  
      - main  
      - develop
```

Le workflow ne se déclenche que pour des `push` sur `main` ou `develop`.

Syntaxe des workflows-Déclencheurs

Les déclencheurs automatiques - filtres

Wildcards possibles :

- `feature/*` → toutes les branches commençant par `feature/` (un seul niveau)
`feature/login`, `feature/api`, ...
- `release/**` → **tout ce qui commence par `feature/`, sur une ou plusieurs profondeurs.**
`feature/login`, `feature/login/ui`, ...

Il est également possible d'exclure certaines branches.

on :

push :

branches :

- `'releases/**'`
- `'!releases/**-alpha'`

Sauf les branches 'releases' qui finissent par « -alpha ».

Syntaxe des workflows-Déclencheurs

Les déclencheurs automatiques - filtres

Les **filtres par chemin** déclenchent le workflow **en fonction des fichiers modifiés** dans un push ou une pull request.

```
on:  
  push:  
    paths:  
      - 'src/**'  
      - '!docs/**'
```

Le workflow se déclenche uniquement si *au moins un* fichier dans `src/` change.

Sauf le dossier 'docs'

```
paths:  
  - '**.md'
```

Se déclenche uniquement si un fichier Markdown est modifié.

Syntaxe des workflows-Déclencheurs

Les déclencheurs programmés

Les **déclencheurs programmés** vous permettent de démarrer un workflow à une heure programmée. Ils utilisent la même syntaxe que les tâches cron.

```
* * * * * commande à exécuter
```

- Jour de la semaine (0 - 7) (dimanche = 0 ou 7)
- Mois (1 - 12)
- Jour du mois (1 - 31)
- Heure (0 - 23)
- Minute (0 - 59)

<https://blog.stephane-robert.info/docs/admin-serveurs/linux/cron/>

Syntaxe des workflows-Déclencheurs

Les déclencheurs programmés

Caractères spéciaux utiles

- * : signifie “à chaque valeur possible”. Par exemple, * * * * * signifie **toutes les minutes**.
- , : permet de **lister plusieurs valeurs**. Ex. : 1,15 dans le champ des jours = le 1er et le 15 du mois.
- - : définit un **intervalle**. Ex. : 1-5 pour du lundi au vendredi.
- / : permet de **définir une fréquence**. Ex. : */2 pour toutes les deux unités.

on:

schedule:

S'exécute toutes les 15 minutes

- cron: '* /15 * * * *'

S'exécute tous les heures de 9 h à 17 h

- cron: '0 9-17 * * *'

S'exécute tous les vendredis à minuit

- cron: '0 0 * * FRI'

S'exécute chaque trimestre à 00 h 00 le premier jour du mois

- cron: '0 0 1 */3 *'

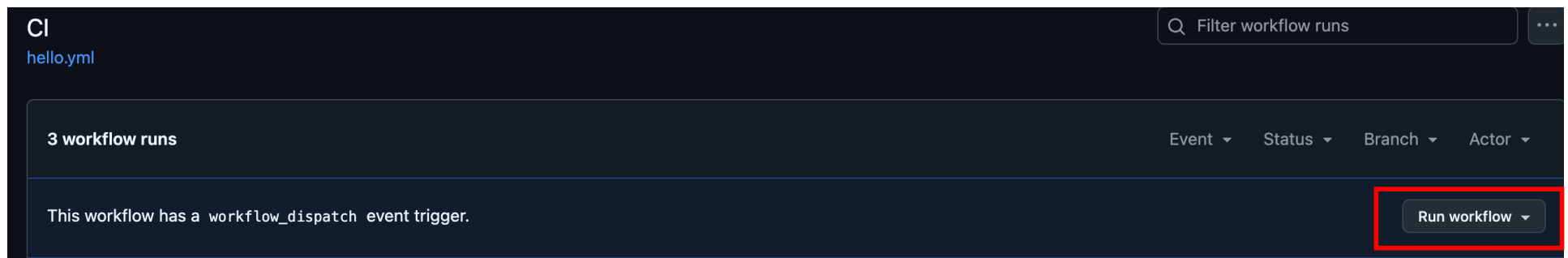
Syntaxe des workflows-Déclencheurs

Les déclencheurs manuels

Les déclencheurs manuels permettent d'exécuter un workflow **manuellement**, directement depuis l'interface GitHub, via le bouton :

Actions → (Workflow) → Run workflow

```
on:  
  workflow_dispatch:
```



Syntaxe des workflows-Déclencheurs

Exercice 1

Modifier ce workflow pour qu'il s'exécute chaque fois que du code est poussé sur une branche (qu'importe laquelle) et également lors d'une pull-request mais uniquement sur la branche *main*.

```
1 name: Exercice 1
2
3 # à compléter
4
5 jobs:
6   job_ex1:
7     runs-on: ubuntu-latest
8
9     steps:
10    - run : echo "Je teste l'exerice 1"
```

Triggered via pull request 2 minutes ago	Status	Total duration	Artifacts
 TRIGianni opened #1 test	Success	<u>6s</u>	—

hello.yml
on: pull_request

 job_ex1 4s

Syntaxe des workflows-Déclencheurs

Exercice 1

Triggered via push 4 minutes ago	Status	Total duration	Artifacts
 TRIGianni pushed ede00a6 test	Success	<u>6s</u>	—

hello.yml

on: push

 job_ex1 2s

Triggered via pull request 2 minutes ago	Status	Total duration	Artifacts
 TRIGianni opened #1 test	Success	<u>6s</u>	—

hello.yml

on: pull_request

 job_ex1 4s

Syntaxe des workflows-Déclencheurs

Exercice 2

Vous devez déclencher le workflow lorsque les fichiers avec une extension « .yaml » sont modifiés lors d'une pull-request, sauf ceux qui ont un dossier parent nommé « test ».

```
1  name: Exercice 2
2
3  # à compléter
4
5  jobs:
6    job_ex1:
7      runs-on: ubuntu-latest
8
9      steps:
10     - run : echo "Je vérifie les fichiers YAML"
```

Syntaxe des workflows-Déclencheurs

Exercice 3

Vous devez déclencher le workflow tous les 5 minutes ou manuellement.

```
1 name: Exercice 3
2
3 # à compléter
4
5 jobs:
6   job_ex1:
7     runs-on: ubuntu-latest
8
9     steps:
10    - run : echo "Je m'exécute tous les 5 minutes"
```

Syntaxe des workflows-Jobs

Les jobs

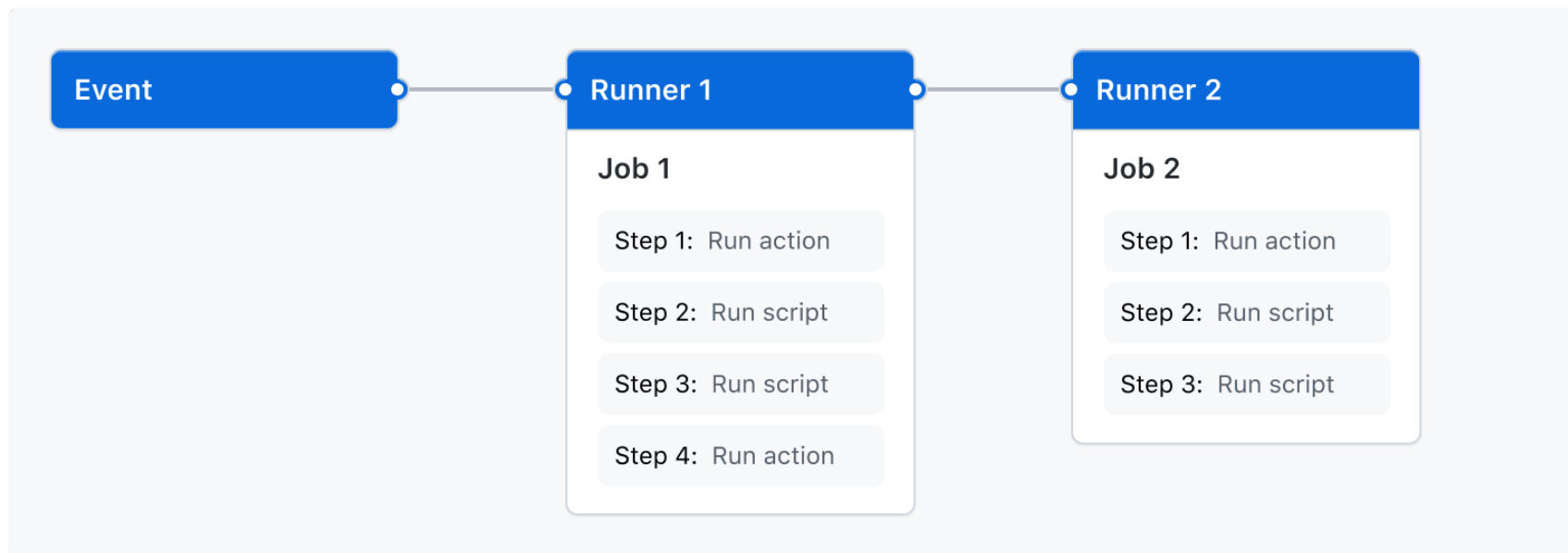
Chaque workflow est composé d'un ou plusieurs jobs (tâches) qui sont un regroupement de tâches. Elles sont exécutées sur un **exécuteur de workflow**, qui peut être une **machine virtuelle** ou un **conteneur** sur lequel un service d'exécution est installé.

GitHub propose des **exécuteurs (runners)** avec les systèmes d'exploitation Linux, macOS et Windows dans différentes tailles de machines, mais vous pouvez également héberger vos propres exécuteurs.

Les jobs s'exécutent en parallèle par défaut.

Une **étape (step)** peut prendre la forme **d'une ligne de commande**, d'un **script** ou d'une étape réutilisable facilement partageable, appelée **GitHub Action** (module).

Syntaxe des workflows-Jobs



Syntaxe des workflows-Jobs

L'exécution d'un job

Chaque workflow est composé d'un ou plusieurs jobs (tâches) qui sont un regroupement de tâches. Elles sont exécutées sur un **exécuteur de workflow**, qui peut être une **machine virtuelle** ou un **conteneur** sur lequel un service d'exécution est installé.

Chaque job est exécuté par un « runner » différent.

Chaque job est composé de plusieurs éléments :

- Une clé, nom simple qui sera affiché si la propriété **name** n'est pas renseignée
- Une propriété **name** (optionnelle), qui ne sert qu'à l'affichage
- Une propriété **runs-on**, qui définit le type de VM sur laquelle sera exécutée le job
- Une propriété **steps**, qui contient un ensemble d'étapes à exécuter

Syntaxe des workflows-Jobs

L'exécution d'un job - Runner

Machine virtuelle	Label
Windows Server 2019	windows-latest
Ubuntu 16.04	ubuntu-16.04
Ubuntu 18.04	ubuntu-18.04
Ubuntu 20.04	ubuntu-20.04
macOS Big Sur 11.0	macos-11.0
macOS Catalina 10.15	macos-10.15

Syntaxe des workflows-Jobs

L'exécution d'un job – Les dépendances

Par défaut, les jobs d'un même workflow sont exécutés en parallèle (démarré en même temps) et indépendamment les uns des autres.

Il existe des cas où l'on aura besoin d'exécuter les jobs de manière séquentielle.

La propriété **needs** permet d'exécuter des jobs de façon **séquentielle** en spécifiant les jobs dont un job dépend.

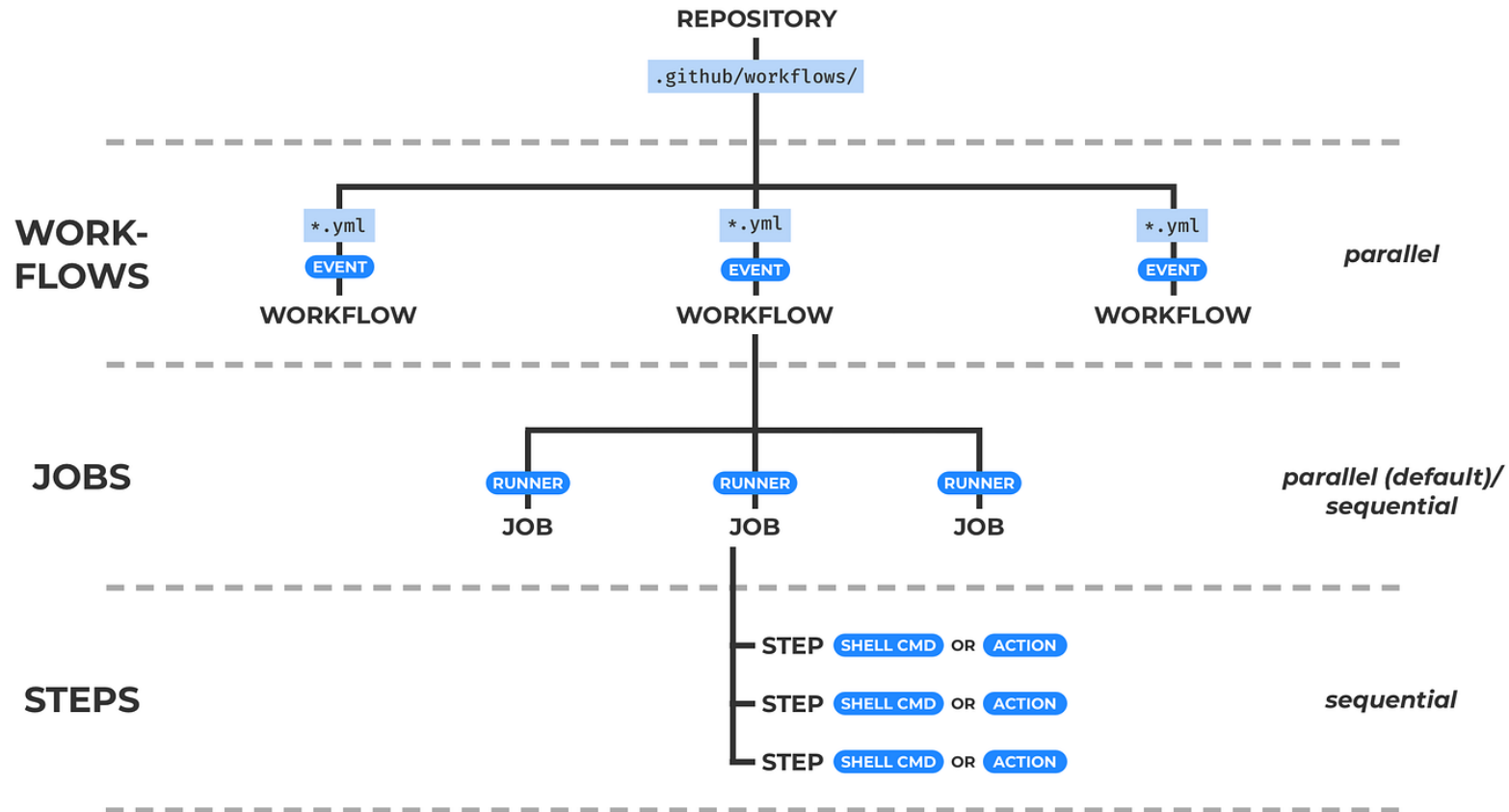
```
jobs:  
  job1:  
  job2:  
    needs: job1  
    if: failure() #indique qu'il s'exécutera seulement si le job1 échoue
```

Syntaxe des workflows-Jobs

L'exécution d'un job – Les dépendances

Vérification	Description
Success()	Lorsque le job précédent a réussi (implicite)
Failure()	Lorsque le job précédent a échoué
Cancelled()	Lorsque le job précédent a été annulé
Always()	Tout le temps

Syntaxe des workflows-Jobs



Syntaxe des workflows-Jobs

Exercice 1

Analyser le workflow suivant et tentez de deviner l'affichage qu'il affichera dans la console.

```
jobs:
  job1:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Bonjour"
  job2:
    needs: job1
    runs-on: ubuntu-latest
    steps:
      - run: echo "Comment"
```

```
job3:
  needs: [job1,job2]
  runs-on: ubuntu-latest
  steps:
    - run: echo "allez"
job4:
  runs-on: ubuntu-latest
  steps:
    - run: echo "vous ? »"
```

Syntaxe des workflows-Steps

Les steps

Le bloc **steps** contient toutes les actions que réalise un workflow.

Une action peut :

- Exécuter une **commande**, une simple ligne de commande ou une suite de commandes
- Exécuter une **Action**, un **module** plus riche et paramétrable créée préalablement et réutilisable à la demande

Les steps – Exécution d’une commande

Il s’agit d’exécuter une ligne de commande console simple et d’en retourner le résultat.

Ce type d’étape se reconnaît par la présence du mot-clé **run**.

Syntaxe des workflows-Steps

Les steps – Exécution d'une commande

```
# façon de faire n°1
steps:
- run: echo 'Hello'
- run: echo 'World'

# façon de faire n°2
steps:
- run: |
    echo 'Hello'
    echo 'World'
```


Syntaxe des workflows-Steps

Les steps – Exécution d'une commande

Il est également courant de donner un alias à chaque step via la propriété **name** (optionnelle) qui permet de faciliter la lecture des logs d'exécution.

```
steps:  
- name: ma première commande  
  run: echo 'Hello'  
- name: ma deuxième commande  
  run: echo 'World'
```

Syntaxe des workflows-Steps

Les steps – Exécution d'une commande

La propriété **shell** permet de spécifier l'interpréteur de la commande.

Supported platform	shell parameter	Description	Command run internally
Linux / macOS	unspecified	The default shell on non-Windows platforms. Note that this runs a different command to when <code>bash</code> is specified explicitly. If <code>bash</code> is not found in the path, this is treated as <code>sh</code> .	<code>bash -e {0}</code>
All	<code>bash</code>	The default shell on non-Windows platforms with a fallback to <code>sh</code> . When specifying a bash shell on Windows, the bash shell included with Git for Windows is used.	<code>bash --noprofile --norc -eo pipefail {0}</code>

<https://docs.github.com/fr/actions/reference/workflows-and-actions/workflow-syntax>

Syntaxe des workflows-Steps

Les steps – Exécution d'une commande

Supported platform	shell parameter	Description	Command run internally
All	<code>pwsh</code>	The Power Shell Core. GitHub appends the extension <code>.ps1</code> to your script name.	<code>pwsh -command ". '{0}'"</code>
All	<code>python</code>	Executes the python command.	<code>python {0}</code>
Linux / macOS	<code>sh</code>	The fallback behavior for non-Windows platforms if no shell is provided and <code>bash</code> is not found in the path.	<code>sh -e {0}</code>
Windows	<code>cmd</code>	GitHub appends the extension <code>.cmd</code> to your script name and substitutes for <code>{0}</code> .	<code>%ComSpec% /D /E:ON /V:OFF /S /C "CALL " {0}"" .</code>

Syntaxe des workflows-Steps

Les steps – Exécution d'une commande

Propriétés supplémentaires :

- **id** : identifiant unique du step lorsque l'on souhaite interagir avec ce dernier tel que récupérer ses variables de sortie
- **with** : pour transmettre des paramètres à la commande/l'action
- **env** : pour déclarer des variables locales au step
- **if**: pour conditionner les exécutions du step
- **continue-on-error**: pour empêcher un step de pouvoir être considéré comme échoué même lorsqu'un problème se présente
- **timeout-minutes**: pour forcer l'arrêt d'un step si celui-ci dépasse la limite de temps autorisée

Syntaxe des workflows-Steps

Les steps – Exécution d'une Action

S'il est possible de pratiquement tout faire uniquement à l'aide de steps **run**, la vraie puissance de GitHub Actions provient de la possibilité d'appeler directement des **modules** de code/script complexes, qui ont été réalisés par de tierces personnes.

L'appel d'une action se fait grâce au mot-clé **uses**, en précisant ensuite le **nom de l'action** à exécuter et sa « **version** ».

La version peut se définir de 3 manières différentes :

- Via la version de l'action : `{owner}/{repo}@v1`
- Via la branche de l'action (cours de développement) : `{owner}/{repo}@branche`
- Via hash(commit) de l'action : `{owner}/{repo}@4ad78`

Syntaxe des workflows-Steps

Les steps – Exécution d'une Action

Il est recommandé d'utiliser des actions autant que possible et de compléter le workflow avec des étapes **run** lorsqu'aucune action ne répond au besoin.

Les steps – Exécution au sein d'un conteneur

```
1 jobs:
2   build:
3     runs-on: ubuntu-latest
4     container: 'node:current'
5
6     steps:
7       - uses: actions/checkout@v1
8       - run: |
9           npm install
10          npm run build
```

La propriété **container** demande au workflow de télécharger l'image docker « node », de démarrer une instance de ce conteneur et d'exécuter en son sein les différentes commandes.

Syntaxe des workflows-Steps

Les steps – Découvrir de nouvelles actions

The screenshot displays the GitHub Actions marketplace. On the left, a sidebar lists categories: Featured, Models, Apps, Actions, and a list of specific actions like AI Assisted, API management, Backup Utilities, Chat, Code quality, Code review, Code Scanning Ready, Code search, Container CI, Continuous integration, Dependency management, and Deployment. The 'Actions' category is selected. The main area is titled 'Actions' with the subtitle 'Automate your workflow from idea to production'. It includes filters for 'Filter: All', 'By: All creators', and 'Sort: Popularity'. A grid of action cards is shown, each with an icon, name, description, and an 'Action' button. The actions listed are: TruffleHog OSS (Find and verify leaked credentials), Metrics embed (An infographics generator), yq - portable yaml processor (create, read, update, delete, merge, validate and do more with yaml), Super-Linter (Super-linter is a ready-to-run collection of linters and code analyzers), Gosec Security Checker (Runs the gosec security checker), Rebuild Armbian and Kernel (Support Amlogic, Rockchip and Allwinner boxes), Checkout (Checkout a Git repository at a particular version), and OpenCommit - improve commits with ... (Replaces lame commit messages with meaningful AI-generated messages).

<https://github.com/marketplace?type=actions>

Syntaxe des workflows-Steps

Exercice

Vous allez créer un workflow GitHub Actions capable :

- d'être lancé manuellement ;
- d'installer la version **18** de Node.js grâce à une action existante (*Marketplace*) : **actions/setup-node@v6.1.0**
- et d'afficher la version installée dans la sortie du workflow.

The screenshot displays the GitHub Actions workflow editor. On the left, the 'Preview' tab is active, showing a workflow file with the following content:

```
# This is a basic workflow to help you get started with Actions

name: CI

# Controls when the workflow will run
on:
  # Triggers the workflow on push or pull request events but only for the "main" branch
  push:
    branches: [ "main" ]
  pull_request:
```

On the right, the 'Marketplace' tab is active, showing a search bar and a section for 'Featured Actions'. The first featured action is 'Setup Node.js environment' by 'actions', which has 4.5k stars. The description for this action is: 'Setup a Node.js environment by adding problem matchers and optionally downloading and adding it to the PATH'.

Syntaxe des workflows-Steps

Leçons interactives : <https://github-actions-hero.vercel.app/lessons/1>

GitHub Actions Hero

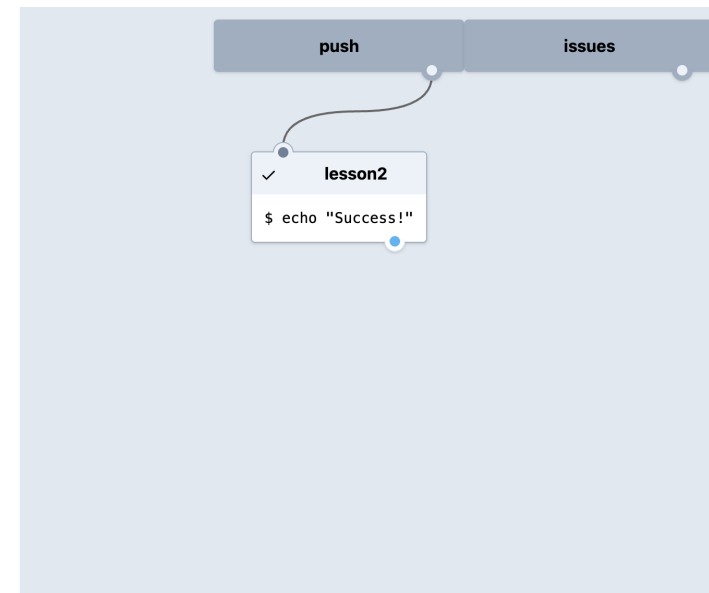
Run for multiple events

Lesson 2

Workflows can also run when any of [multiple events](#) occur.

Currently this workflow runs on every `push`, update it so that it also runs whenever any of the `issues` in the repository is modified.

```
1 name: Multiple Events
2
3 on: [push]
4
5 jobs:
6   lesson2:
7     runs-on: ubuntu-latest
8     steps:
9     - run: echo "Success!"
```



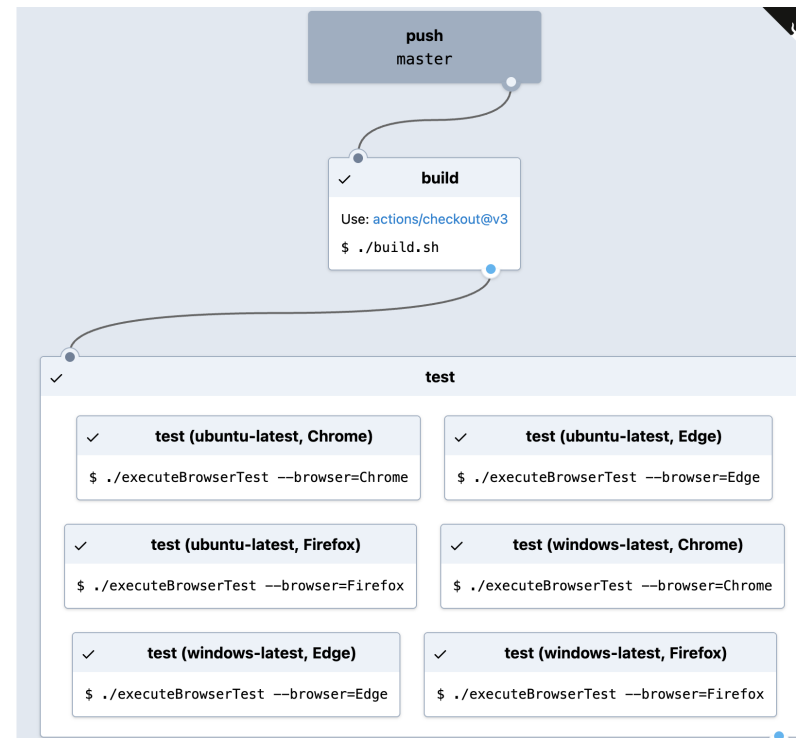
Syntaxe des workflows-Steps

Terrain de jeux: <https://github-actions-hero.vercel.app/playground>

GitHub Actions Hero

Playground

```
1 name: Matrix
2
3 on: [push]
4
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v3
10      - run: ./build.sh
11
12   test:
13     needs: build
14     runs-on: ${{ matrix.os }}
15     strategy:
16       matrix:
17         os: [ubuntu-latest, windows-latest]
18         browser: [Chrome, Edge, Firefox]
19     steps:
20       - run: ./executeBrowserTest --browser=${{ matrix.browser }}
21
22   deploy:
23     needs: test
24     runs-on: ubuntu-latest
25     steps:
26       - run: ./uploadRelease.sh
27
```



Syntaxe des workflows-Matrices

Les matrices

On pourrait avoir la nécessité de compiler un même code source, avec différents paramètres de compilation, par exemple une fonctionnalité activée ou non ou simplement de compiler la même application sur plusieurs versions du framework sous-jacent.

⇒ **Conséquence** : fichier trop long et de nombreuses lignes de code dupliquées

⇒ **Solution** : utilisation des matrices

Les matrices permettent de générer dynamiquement plusieurs jobs à partir d'une seule définition.

Les matrices - déclaration

Elles se déclarent au niveau d'un job, comme propriété d'une strategy et puis d'une sous-propriété nommée matrix ainsi d'un tableau de type « **clé: [valeur1, valeur2]** ».

Les variables sont accessibles sous la forme **`${{matrix.clé}}`**

Syntaxe des workflows-Matrices

GitHub Actions Hero

Playground

Workflow: Matrix ▾

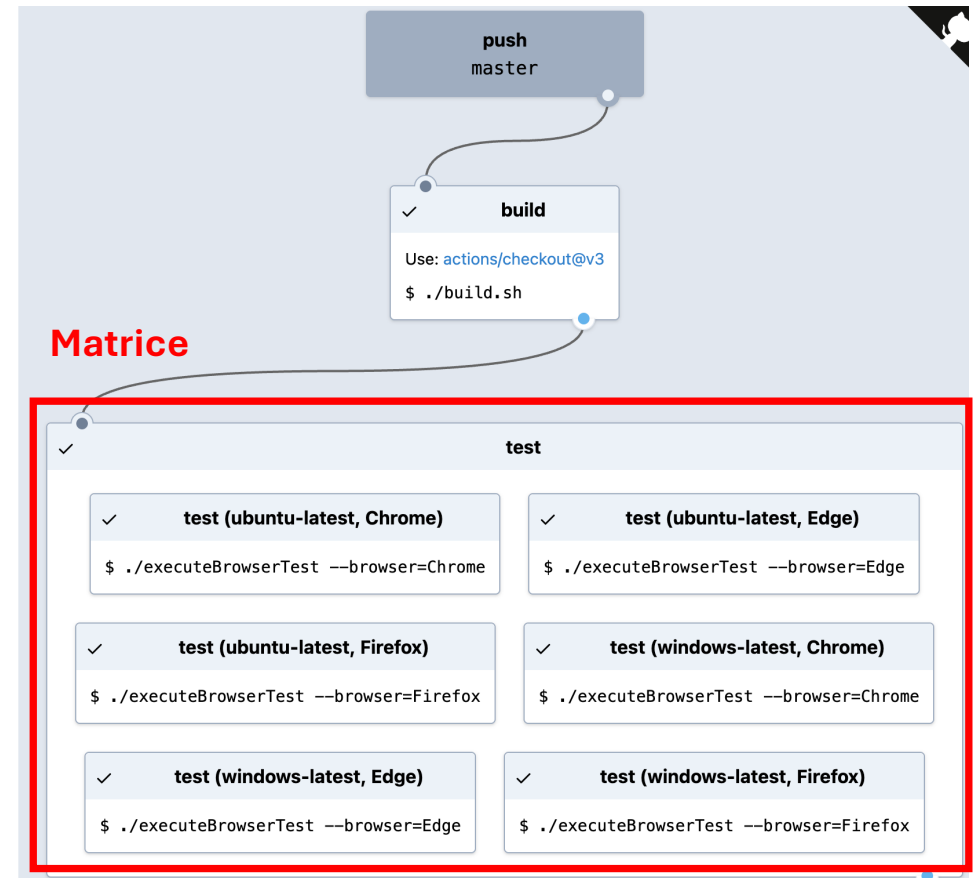


```

1 name: Matrix
2
3 on: [push]
4
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     steps:
9       - uses: actions/checkout@v3
10      - run: ./build.sh
11
12   test:
13     needs: build
14     runs-on: ${{ matrix.os }}
15     strategy:
16       matrix:
17         os: [ubuntu-latest, windows-latest]
18         browser: [Chrome, Edge, Firefox]
19     steps:
20       - run: ./executeBrowserTest --browser=${{ matrix.browser }}
21
22   deploy:
23     needs: test
24     runs-on: ubuntu-latest
25     steps:
26       - run: ./uploadRelease.sh
27

```

Matrice



Syntaxe des workflows-Matrices

Les matrices - exclude

L'option **exclude** permet de **retirer certaines combinaisons** que vous ne souhaitez pas exécuter.

```
12 test:
13   needs: build
14   runs-on: ${ matrix.os }
15   strategy:
16     matrix:
17       os: [ubuntu-latest, windows-latest]
18       browser: [Chrome, Edge, Firefox]
19       exclude:
20         - os: windows-latest
21           browser: Edge
22   steps:
23     - run: ./executeBrowserTest --browser=${ matrix.browser }
24
```

Syntaxe des workflows-Matrices

Les matrices - include

L'option **include** permet d'**ajouter des combinaisons supplémentaires** ou d'**ajouter des variables spécifiques qui ne sont pas générées automatiquement** par la matrice.

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    node: [18, 20]
```

GitHub exécute 4 combinaisons

```
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    node: [18, 20]
  include:
    - os: ubuntu-latest
      node: 22
```

Ajouter **Node 22 uniquement pour Ubuntu**, sans l'ajouter à la matrice globale.

GitHub ajoute la combinaison `ubuntu + node 22`.

Syntaxe des workflows-Variables

Les variables

Les variables permettent de définir des clés possédant des valeurs qui seront réutilisable au sein du workflow.

Les variables - déclaration

La déclaration d'une variable se fait simplement en ajoutant la propriété **env** puis en déclarant des clés-valeurs .

```
env:  
  MA_VARIABLE_PRENOM: "Alice"  
  MA_VARIABLE_NOM: "Bob"  
  MA_VARIABLE_MULTILIGNE: |  
                           "Bonjour  
                           à tous  
                           !"
```

Syntaxe des workflows-Variables

Les variables

Les variables permettent de définir des clés possédant des valeurs qui seront réutilisable au sein du workflow.

Les variables - déclaration

La déclaration d'une variable se fait simplement en ajoutant la propriété **env** puis en déclarant des clés-valeurs .

```
env:  
  MA_VARIABLE_PRENOM: "Alice"  
  MA_VARIABLE_NOM: "Bob"  
  MA_VARIABLE_MULTILIGNE: |  
                          "Bonjour  
                           à tous  
                           !"
```

Les noms des variables sont sensibles à la casse.

Syntaxe des workflows-Variables

Les variables - Utilisation

```
${{ env.MA_VARIABLE_PRENOM }}
```

```
env:  
  MA_VARIABLE_PRENOM: "Alice"  
steps:  
-run: echo ${{ MA_VARIABLE_PRENOM }}
```

Les variables – Déclaration dynamique

Parfois la variable n'est pas connue à l'avance où celle-ci sera définie au cours du workflow, pour être utilisée ou transmise à un autre step.

Syntaxe des workflows-Variables

Les variables - Déclaration dynamique

```
job:
  runs-on: ubuntu-latest
  env:
    VARIABLE: jaune
  steps:
  - run: echo $VARIABLE
  - run: |
      echo "VARIABLE=vert" >> $GITHUB_ENV
      echo $VARIABLE
  - run: echo $VARIABLE
```

Transférer »VARIABLE=vert« dans la variable globale \$GITHUB_ENV

Syntaxe des workflows-Variables

Les variables de sorties

Une variable de sortie est une variable associée à un step particulier mais étant visible depuis d'autres steps.

La définition d'une variable de sortie : **set-output**

```
steps:  
- id: monStep  
  run: echo "::set-output name=NOMVARIABLE::hello"  
- run: echo **${{ steps.monStep.outputs.NOMVARIABLE }}**
```

Les variables d'environnement prédéfinies

Syntaxe des workflows-Variables

Les variables d'environnement prédéfinies

GITHUB_REPOSITORY

Contient : owner/repo

Exemple : octocat/hello-world

GITHUB_ACTOR

Nom de l'utilisateur qui a déclenché le workflow.

Exemple : octocat

GITHUB_SHA

Le commit SHA qui a déclenché le workflow.

Exemple : d1e2f3c4b5...

Syntaxe des workflows-Variables

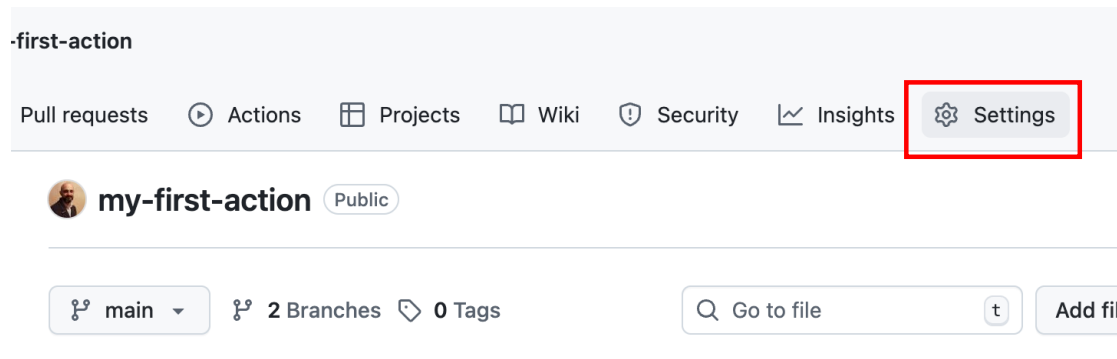
Exercices

Chercher sur la marketplace l'action **get-current-time**. Utilisez cette action dans un workflow et puis affichez dans la console la date actuelle au format français.

Syntaxe des workflows-Secrets

Les secrets

Les secrets sont l'équivalent de variables d'environnements, mais chiffrés et stockés en dehors du fichier YAML de workflow.



Syntaxe des workflows-Secrets

Les secrets - Création

 Codespaces

 Pages

Security

 Advanced Security

 Deploy keys

 **Secrets and variables** ^

Actions

Codespaces

Repository secrets

This repository has no secrets.

New repository secret

Syntaxe des workflows-Secrets

Les secrets - Création

Actions secrets / New secret

Name *

YOUR_SECRET_NAME



Secret *

Add secret

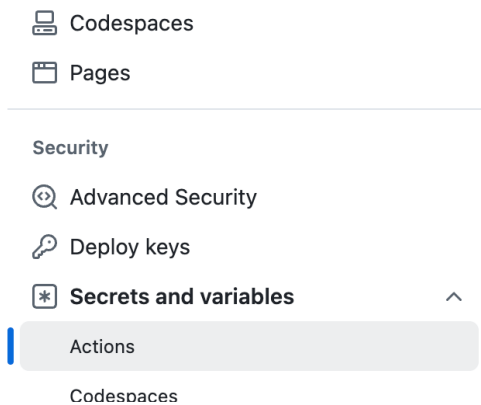


WALLONIE-BRUXELLES
ENSEIGNEMENT



Syntaxe des workflows-Secrets

Les secrets - Création



Repository secrets

New repository secret

Name	Last updated
MON_SECRET	now

Les secrets - Utilisation

steps:

– run: echo "{{ secrets.MONSECRET }}"

Syntaxe des workflows-Jobs

Solution : Exercice job – ex1

Bonjour vous ? Comment allez

